COMPUTER ENGINEERING

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Sep 10, 2026	**DATE SUBMITTED:**	**Sep 15, 2026**
NAME:	Yazinel Corro	**INSTRUCTOR:**	Engr.Jimlord Quejado

ULY

ACTIVITY NO. 8.1 **Hands-on Activity 8.1 Sorting Algorithms Pt 2**

1. PROCEDURE OUTPUT

ILO A

Main.cpp

```cpp
#include <iostream>
#include "sortingAlgo2.h"
#include <cstdlib>
#include <ctime>

const int maxSize = 100;

void displayArray(int arr[]);

int main() {
    int array[maxSize];

    std::srand(std::time(0));

    for (int i = 0; i < maxSize; i++){
        array[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArray(array);

    /*
    std::cout << "\nShell Sort:\n";
    shellSort(array, maxSize);
    displayArray(array);
    */

    /*
    std::cout << "\nMerge Sort:\n";
    mergeSort(array, 0, maxSize - 1);
    displayArray(array);
    */


    std::cout << "\nQuick Sort:\n";
    quickSort(array, 0, maxSize - 1);
```

```
    std::cout << "\nQuick Sort:\n";
    quickSort(array, 0, maxSize - 1);
    displayArray(array);


}

void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

SortingAlgo2.h

```cpp
#ifndef SORTINGALAGO2_H
#define SORTINGALAGO2_H

//Shell Sort
template <typename T>
void shellSort (T arr[], int size) {
    for (int i = size / 2; i > 0; i /= 2){
        for (int j = i; j < size; j++){
            T temp = arr[j];
            int k;
            for (k = j; k >= i && arr[k - i] > temp; k -= i){
                arr[k] = arr[k - i];
            }
            arr[k] = temp;
        }
    }
}

//helper function for merge sort
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0;
    int j = 0;
    int k = left;

```

```
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

//Merge Sort
template <typename T>
void mergeSort (T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
```

```
1   #ifndef SORTINGALAGO2_H
64  //Merge Sort
65  template <typename T>
66  void mergeSort (T arr[], int left, int right) {
67      if (left < right) {
68          int mid = left + (right - left) / 2;
69          mergeSort(arr, left, mid);
70          mergeSort(arr, mid + 1, right);
71          merge(arr, left, mid, right);
72      }
73  }
74
75  //helper function for quick sort
76  template <typename T>
77  int partition(T arr[], int low, int high) {
78      T pivot = arr[high];
79      int i = low - 1;
80
81      for (int j = low; j < high; j++) {
82          if (arr[j] <= pivot) {
83              i++;
84              std::swap(arr[i], arr[j]);
85          }
86      }
87      std::swap(arr[i + 1], arr[high]);
88      return i + 1;
89  }
90
91  //quickSort
92  template <typename T>
93  void quickSort(T arr[], int low, int high) {
94      if (low < high) {
95          int pi = partition(arr, low, high);
96          quickSort(arr, low, pi - 1);
97          quickSort(arr, pi + 1, high);
98      }
```

```
91  //quickSort
92  template <typename T>
93  void quickSort(T arr[], int low, int high) {
94      if (low < high) {
95          int pi = partition(arr, low, high);
96          quickSort(arr, low, pi - 1);
97          quickSort(arr, pi + 1, high);
98      }
99  }
100
101
102 #endif
```

Output

```
Unsorted Array:
51 24 0 63 2 92 62 57 58 64 9 27 12 83 76 36 38 6 83 89 70 99 93 2 29 89 80 29 30 50 23 48 79 82 15 5 85 21 12 60 3
 82 75 44 8 92 0 89 81 0 24 6 38 85 86 32 79 80 80 63 51 44 62 77 18 36 31 31 16 63 72 6 9 12 33 45 39 89 26 97 15
51 13 32 35 68 71 52 69 92 70 53 23 19 17 71 40 41

Shell Sort:
0 0 0 2 2 5 6 6 6 6 8 9 9 12 12 12 13 15 15 16 17 18 19 21 23 23 24 24 26 27 29 29 30 30 31 31 32 32 33 35 36 36 38
 39 40 41 44 44 45 48 48 50 51 51 51 52 53 57 58 60 62 62 63 63 63 64 68 69 70 70 71 71 72 75 76 77 79 79 80 80 80
82 82 83 83 85 85 86 89 89 89 89 92 92 92 93 97 99

Merge Sort:
0 0 0 2 2 5 6 6 6 6 8 9 9 12 12 12 13 15 15 16 17 18 19 21 23 23 24 24 26 27 29 29 30 30 31 31 32 32 33 35 36 36 38
 39 40 41 44 44 45 48 48 50 51 51 51 52 53 57 58 60 62 62 63 63 63 64 68 69 70 70 71 71 72 75 76 77 79 79 80 80 80
82 82 83 83 85 85 86 89 89 89 89 92 92 92 93 97 99

Quick Sort:
0 0 0 2 2 5 6 6 6 6 8 9 9 12 12 12 13 15 15 16 17 18 19 21 23 23 24 24 26 27 29 29 30 30 31 31 32 32 33 35 36 36 38
 39 40 41 44 44 45 48 48 50 51 51 51 52 53 57 58 60 62 62 63 63 63 64 68 69 70 70 71 71 72 75 76 77 79 79 80 80 80
82 82 83 83 85 85 86 89 89 89 89 92 92 92 93 97 99
PS C:\Users\yazinel\OneDrive\Desktop\cpe 010 - corro-hoa1\HOA8p1\procedure>
```

Code + Console Screenshot	
Observations	My observation is that the array contains random numbers from 0 to 99, so the values are not arranged in order. This array is used as the starting data for testing the sorting algorithms.

```
6   const int maxSize = 100;
7
8   void displayArray(int arr[]);
9
10  int main() {
11      int array[maxSize];
12
13      std::srand(std::time(0));
14
15      for (int i = 0; i < maxSize; i++){
16          array[i] = std::rand() % 100;
17      }
18
19      std::cout << "Unsorted Array:\n";
20      displayArray(array);
```

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	Shell Sort arranged the random values from smallest to largest. It uses gaps to make the sorting process faster than simply comparing nearby values.

```
std::cout << "\nShell Sort:\n";
shellSort(array, maxSize);
displayArray(array);
```

Table 8-2. Shell Sort Technique

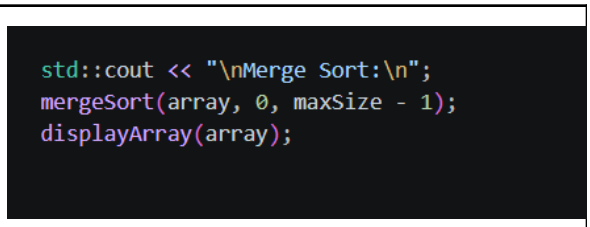
Code + Console Screenshot	
Observations	Merge Sort successfully arranged all the values in ascending order. It works by dividing the array into smaller sections and merging them in the correct order.

Table 8-3. Merge Sort Algorithm

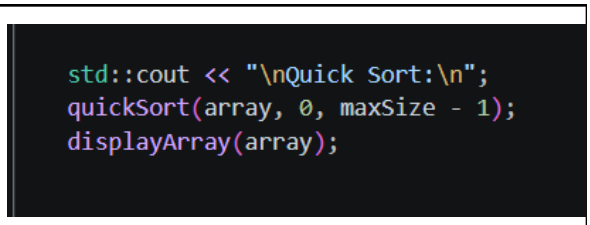
Code + Console Screenshot	
Observations	Quick Sort successfully sorted the array from smallest to largest. It uses a pivot to divide the values into smaller groups and sorts each group.

Table 8-4. Quick Sort Algorithm

.

SUPPLEMENTARY ACTIVITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

CODE

Main (PROB 1)

procedure > suppleprob1 > main.cpp > main()

```cpp
#include <iostream>
#include "sortAlgo1.h"


const int maxSize = 15;

void displayArray(int arr[]);

int main() {

    int array[maxSize] = {
        8, 3, 7, 2, 9,
        5, 1, 6, 4, 10,
        12, 11, 15, 13, 14
    };

    std::cout << "Original Array:\n";
    displayArray(array);

    std::cout << "\nQuick Sort with Insertion Sort:\n";

    quickSortWithInsertion(array, 0, maxSize - 1);

    displayArray(array);

    return 0;
}

void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}
```

Sorting Algo (prob 1)

```cpp
#ifndef SORTALGO2_H
#define SORTALGO2_H

#include <iostream>
#include <algorithm>

// Insertion Sort
template <typename T>
void insertionSort(T arr[], int low, int high) {
    for (int i = low + 1; i <= high; i++) {
        T temp = arr[i];
        int j = i - 1;

        while (j >= low && arr[j] > temp) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = temp;
    }
}

// Helper function for Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);
```

```
35
36      std::swap(arr[i + 1], arr[high]);
37
38      return i + 1;
39  }
40
41  // Quick Sort using Insertion Sort for the sublists
42  template <typename T>
43  void quickSortWithInsertion(T arr[], int low, int high) {
44
45      if (low < high) {
46
47          int pi = partition(arr, low, high);
48
49          // Sort the left sublist using Insertion Sort
50          insertionSort(arr, low, pi - 1);
51
52          // Sort the right sublist using Insertion Sort
53          insertionSort(arr, pi + 1, high);
54      }
55  }
56
57  #endif
```

OUTPUT OF THE CODE

```
Original Array:
8 3 7 2 9 5 1 6 4 10 12 11 15 13 14

Quick Sort with Insertion Sort:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
PS C:\Users\yazinel\OneDrive\Desktop\cpe 010 - corro-
```

Main (PROB 2)

```cpp
#include <iostream>
#include "sortAlgo2.h"

const int maxSize = 15;

void displayArray(int arr[]);

int main() {

    int array[maxSize] = {
        4, 34, 29, 48, 53,
        87, 12, 30, 44, 25,
        93, 67, 43, 19, 74
    };

    std::cout << "Unsorted Array:\n";
    displayArray(array);

    std::cout << "\nQuick Sort:\n";
    quickSort(array, 0, maxSize - 1);
    displayArray(array);

    return 0;
}

void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}
```

Sorting Algo (PROB 2)

```cpp
#ifndef SORTINGALAGO2_H
#define SORTINGALAGO2_H

#include <iostream>
#include <algorithm>

// Helper function for Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);

    return i + 1;
}

// Quick Sort
template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
#endif
```

OUTPUT

```
Unsorted Array:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74

Quick Sort:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
PS C:\Users\yazinel\OneDrive\Desktop\cpe 010 - corro-hoa1\HOA8p1\suppleprob2>
```

CONCLUSIONS

Summary of lessons learned:

- This activity demonstrated the efficiency of advanced sorting algorithms. Shell Sort, Merge Sort, and Quick Sort in organizing randomized datasets using gap sequences and divide-and-conquer strategies. It also showed that hybrid approaches, such as using Insertion Sort on Quick Sort sublists, are functionally valid for array sorting.

Analysis of the procedure:

- The procedure successfully generated a 100-element randomized array containing integers from 0 to 99 and passed it through each sorting algorithm. All three methods—Shell, Merge, and Quick Sort—correctly reordered the data into ascending order, validating their algorithmic logic

Analysis of the supplementary activity:

- The first problem proved that you can use Insertion Sort to arrange the smaller sublists created by Quick Sort's partition step. The second problem showed that Merge Sort and Quick Sort achieve speed because they split the list in half repeatedly while doing total work at each level. Overall, Quick Sort is often the fastest choice for random numbers because it sorts elements directly in place without extra setup.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- Overall, I feel like I did okay despite finding the recursive logic and sublist management really challenging to implement. My main area for improvement is spending more time tracing recursive algorithms on paper before jumping straight into coding.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGpt

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.